

Respostas Trabalho1 IA 2025

Manuel Pereira- 45855

Miguel Aleixo- 51653

Mariana Cavaco- 51820

- a) **Represente em Prolog o estado inicial e o estado final para cada um dos dois exemplos**

Exemplo 1:

```
estado_inicial(1,1,[]).  
estado_final(2,7,[a,b]).
```

Exemplo 2:

```
estado_inicial(1,1,[]).  
estado_final(5,7,[a,b]).
```

- b) **Represente em Prolog os operadores de transição de estados para este problema.**

```
% Atualiza lista de objetos ao entrar em uma célula  
atualiza_objetos(X, Y, Objs, NovosObjs) :-  
    ( objeto_a(X,Y), \+ member(a, Objs) -> NovosObjs = [a|Objs]  
    ; objeto_b(X,Y), \+ member(b, Objs) -> NovosObjs = [b|Objs]  
    ; NovosObjs = Objs  
    ).  
  
% Movimentos (Cima, Baixo, Esquerda, Direita)  
movimento((X,Y,Objs), (X,Y1,NovosObjs)) :-  
    Y1 is Y + 1,  
    \+ obstaculo(X, Y1),  
    atualiza_objetos(X, Y1, Objs, NovosObjs).  
  
movimento((X,Y,Objs), (X,Y1,NovosObjs)) :-  
    Y1 is Y - 1,  
    \+ obstaculo(X, Y1),  
    atualiza_objetos(X, Y1, Objs, NovosObjs).  
  
movimento((X,Y,Objs), (X1,Y,NovosObjs)) :-  
    X1 is X - 1,  
    \+ obstaculo(X1, Y),  
    atualiza_objetos(X1, Y, Objs, NovosObjs).  
  
movimento((X,Y,Objs), (X1,Y,NovosObjs)) :-  
    X1 is X + 1,  
    \+ obstaculo(X1, Y),
```

```
atualiza_objetos(X1, Y, Objs, NovosObjs).
```

c) Apresente o código em Prolog do algoritmo de pesquisa não informada mais eficiente a resolver este problema

```
% BFS - Busca em Largura (ponto de entrada)
bfs(CaminhoFinal) :-
    estado_inicial(X, Y, Objs),
    bfs([[ (X, Y, Objs) ]], [ (X, Y, Objs) ], CaminhoFinal).

bfs([[EstadoAtual|Caminho]|_], _, Solucao) :-
    verifica_estado_final(EstadoAtual),
    reverse([EstadoAtual|Caminho], Solucao).

bfs([Caminho|Fila], Visitados, Solucao) :-
    Caminho = [EstadoAtual|_],
    findall(
        [NovoEstado|Caminho],
        ( movimento(EstadoAtual, NovoEstado),
          \+ member(NovoEstado, Caminho),
          \+ member(NovoEstado, Visitados)
        ),
        NovosCaminhos
    ),
    findall(
        Estado,
        (member([Estado|_], NovosCaminhos))
    ),
    EstadosNovos
),
append(Visitados, EstadosNovos, NovoVisitados),
append(Fila, NovosCaminhos, NovaFila),
bfs(NovaFila, NovoVisitados, Solucao).
```

A escolha do **BFS (Busca em Largura)** justifica-se pela sua garantia de encontrar sempre o caminho mais curto quando o custo dos movimentos é uniforme, como acontece neste problema. Além disso, é um algoritmo simples, determinístico e independente de heurísticas, o que evita possíveis falhas decorrentes de uma má definição das mesmas. BFS também assegura uma exploração completa do espaço de estados, permitindo validar a existência de múltiplas soluções e comparar diferentes caminhos. Embora o seu consumo de memória possa ser elevado em problemas com um grande espaço de estados, esta abordagem é viável quando o número total de estados é gerenciável, garantindo uma solução ótima sem necessidade de otimização avançada.

No contexto deste problema, estimamos que o BFS visite cerca de **150 a 200 estados**, dependendo da disposição dos obstáculos e da necessidade de recolher os objetos obrigatórios antes de alcançar o estado final. Em termos de memória, o pior caso ocorre quando o BFS mantém na fronteira um nível inteiro da árvore de busca, o que pode significar armazenar simultaneamente cerca de **30 a 50 estados**. Estas estimativas dependem da distribuição dos estados acessíveis e da profundidade da solução ótima, mas refletem a escalabilidade do algoritmo dentro deste espaço de busca.

d) Indique:

- i. qual o **número total (exacto) de estados visitados**,
- ii. qual o **máximo número (exacto) de estados que têm que estar simultaneamente em memória**.

Exemplo 1:

EstadosVisitados = 137
MaxMemoria = 35

Exemplo 2:

EstadosVisitados = 509
MaxMemoria = 107

e) **Proponha duas heurísticas admissíveis para estimar o custo de um estado até à solução para este problema.**

Duas heurísticas admissíveis para estimar o custo de um estado até a solução deste problema são: (1) a **distância de Manhattan até a posição final**, que representa o menor número de movimentos necessários para alcançar a posição final, assumindo que não há obstáculos nem a necessidade de recolher objetos. Esta heurística é admissível porque nunca subestima o custo real do caminho, uma vez que apenas considera deslocamentos horizontais e verticais diretos. (2) A **distância de Manhattan até o objeto mais distante que ainda precisa ser recolhido, somada à distância desse objeto até a posição final**. Esta heurística tem em conta que o agente deve recolher os objetos antes de alcançar o estado final, garantindo um custo mínimo estimado para esse percurso. Ambas as heurísticas são admissíveis porque nunca superestimam o custo real da solução: a primeira fornece um limite inferior do número de passos necessários sem considerar obstáculos, enquanto a segunda adiciona a necessidade de recolha dos objetos, mas sem contar possíveis desvios causados por obstáculos.

f) Apresente o código em Prolog do algoritmo de pesquisa informada mais eficiente para resolver os 2 exemplos deste problema usando as heurísticas definidas na alínea anterior.

```
aestrela(CaminhoFinal, EstadosVisitados, MaxFronteira) :-
    estado_inicial(X, Y, Objetos),
    h2((X, Y, Objetos), H),
    aestrela_busca([(X, Y, Objetos), [(X, Y, Objetos)], 0, H], 0, 1,
CaminhoReverso, EstadosVisitados, MaxFronteira),
    reverse(CaminhoReverso, CaminhoFinal).
aestrela_busca([(X, Y, Objetos), Caminho, _, _]|_, Visitados,
MaxFronteira, Caminho, Visitados, MaxFronteira) :-
    verifica_estado_final((X, Y, Objetos)).
aestrela_busca([(X, Y, Objetos), Caminho, G, _]|Fila, Visitados,
MaxFronteiraAtual, Solucao, VisitadosFinais, MaxFronteiraFinal) :-
    findall(((NX, NY, NObjs), [(NX, NY, NObjs) | Caminho], NG, NF),
        (movimento((X, Y, Objetos), (NX, NY, NObjs)),
            \+ member((NX, NY, NObjs), Caminho), % Evitar ciclos
            NG is G + 1,
            h2((NX, NY, NObjs), H),
            NF is NG + H
        ),
        NovosNos),
    append(Fila, NovosNos, NovaFila),
    length(NovaFila, TamanhoFila), % Obtém tamanho da nova fronteira
    MaxFronteiraNova is max(MaxFronteiraAtual, TamanhoFila),
    EstadosVisitadosNovo is Visitados + 1,
    ordenar_por_f(NovaFila, FilaOrdenada),
    aestrela_busca(FilaOrdenada, EstadosVisitadosNovo, MaxFronteiraNova,
Solucao, VisitadosFinais, MaxFronteiraFinal).
ordenar_por_f(Lista, ListaOrdenada) :-
    criar_pares(Lista, ListaPares),
    sort(ListaPares, ListaParesOrdenada), % Ordena por F
    remover_pares(ListaParesOrdenada, ListaOrdenada).
criar_pares([], []).
criar_pares([(X, Y, Objs), Caminho, G, F) | Resto], [(F, (X, Y, Objs),
Caminho, G, F) | Pares]) :-
    criar_pares(Resto, Pares).

remover_pares([], []).
remover_pares([_, X, Caminho, G, F) | Resto], [(X, Caminho, G, F) |
Lista]) :-
```

```
remover_pares(Resto, Lista).
```

Para resolver este problema da forma mais eficiente, devemos utilizar **A***, *um algoritmo de pesquisa informada que garante encontrar o caminho ótimo se a heurística for admissível e consistente*. A escolha de **A*** deve-se ao facto de este explorar os nós com menor custo total **f(n)=g(n)+h(n)** onde:

- **g(n)** representa o custo real acumulado desde o estado inicial até ao estado atual.
- **h(n)** é a heurística escolhida, que estima o custo restante até ao estado final.

Dado que definimos duas heurísticas admissíveis anteriormente, a heurística mais eficiente para este problema será **h₂**, pois tem em conta tanto a distância até os objetos necessários como a distância destes até o objetivo. Esta heurística reduz significativamente o espaço de busca ao considerar a necessidade de recolher os objetos antes de concluir a tarefa.

g) Depois de resolver os 2 exemplos deste problema com o algoritmo da alínea anterior indique para cada função heurística:

- qual o número total (exacto) de estados visitados,**
- qual o máximo número (exacto) de estados que têm que estar simultaneamente em memória**

Número de estados visitados no exemplo 1: 7

Número de estados visitados no exemplo 2: 73

Número máximo de estados simultaneamente em memória, exemplo 1: 13

Número máximo de estados simultaneamente em memória, exemplo 2: **121**